

Lesson: Exploring Clustering Techniques

Introduction

Imagine sorting a giant pile of unlabeled photos into meaningful groups—maybe by scenery, people, or colors—without any instructions. That’s the magic of **clustering techniques** like k-means and hierarchical clustering! In this lesson, you’ll get hands-on with these unsupervised learning methods, learning how k-means organizes data into tidy clusters, how hierarchical clustering creates a tree of relationships with different linkage methods, and when to use each. With math and code to guide you, you’ll be ready to uncover patterns in data like a pro!

Learning Outcomes

By the end of this lesson, you will be able to:

1. **Apply** k-means clustering and **select** the optimal number of clusters for a dataset.
 2. **Describe** hierarchical clustering, its types, and **interpret** dendrograms to understand data relationships.
 3. **Compare** the advantages of k-means and hierarchical clustering for different scenarios.
-

Understanding K-Means Clustering

K-means clustering groups data points into **k** clusters by assigning each point to the cluster with the closest centroid. It’s like sorting candies into jars, minimizing the distance between candies and their jar’s center. The steps are:

1. Choose **k**, the number of clusters.
2. Randomly initialize **k** centroids.
3. Assign each point to the nearest centroid.
4. Recalculate centroids as the mean of points in each cluster.
5. Repeat until centroids stabilize (convergence).

The objective is to minimize the **within-cluster sum of squares (WCSS)**:

$$WCSS = \sum_{i=1}^n \sum_{k=1}^K \|x_i - \mu_k\|^2$$

where x_i is a data point, μ_k is the centroid of cluster k , and n is the number of points.

Here’s a Python snippet using scikit-learn to run k-means on the Iris dataset:

```

import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn import datasets

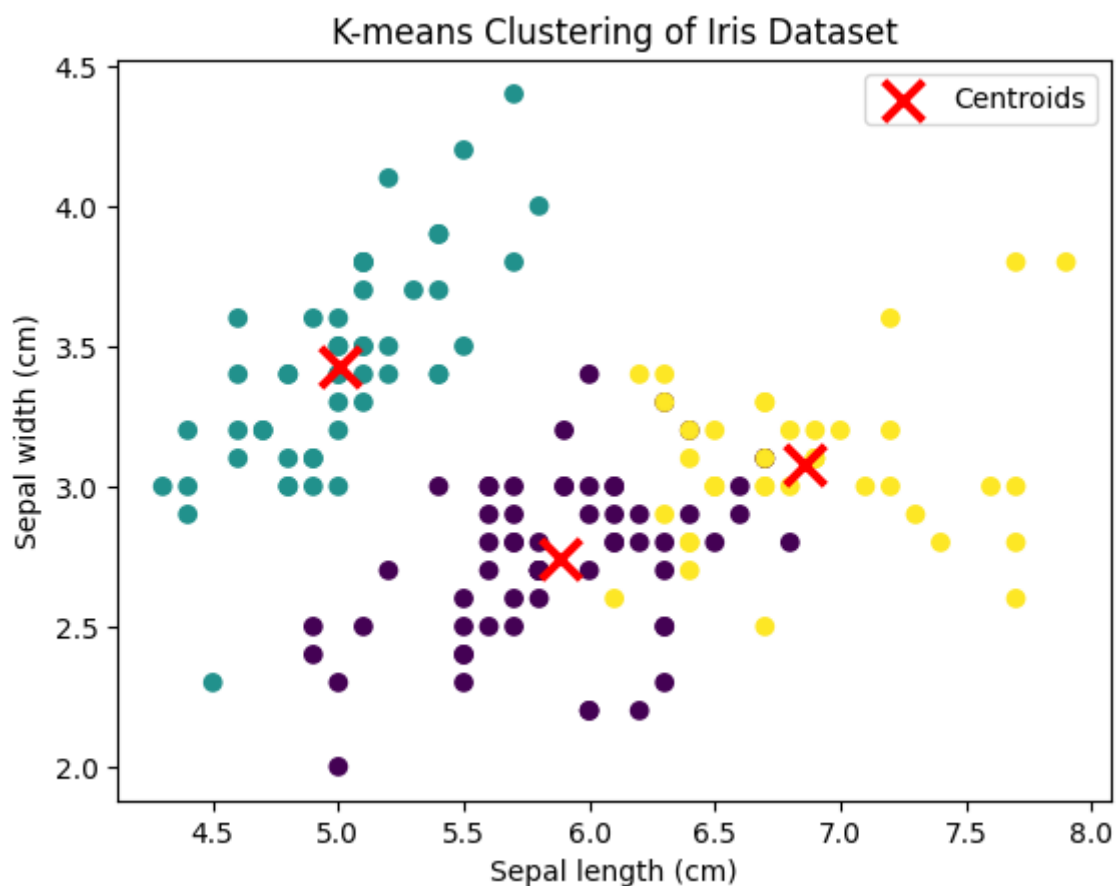
# Load the Iris dataset
iris = datasets.load_iris()
iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Choose number of clusters
k = 3

# Create and fit KMeans model
kmeans = KMeans(n_clusters=k, random_state=0)
kmeans.fit(iris_df)

# Get cluster labels
labels = kmeans.labels_
print("Cluster labels:", labels[:10])

```



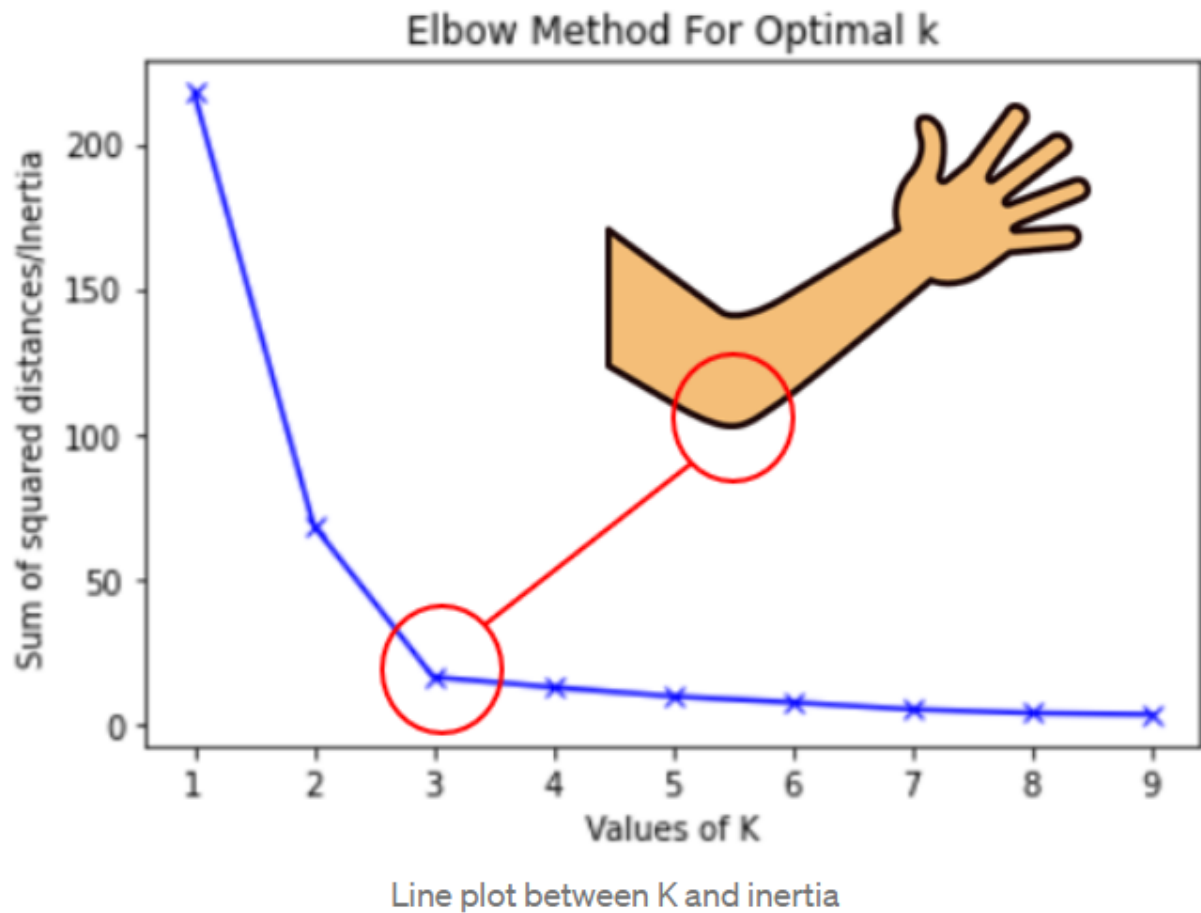
Note: Convergence occurs when centroids stop moving significantly, indicating stable clusters.

Choosing the Right Number of Clusters

Selecting **k** is key. The **elbow method** helps by plotting WCSS for various **k** values and finding the "elbow" where WCSS stops dropping sharply:

$$WCSS = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

Here's an example elbow plot:

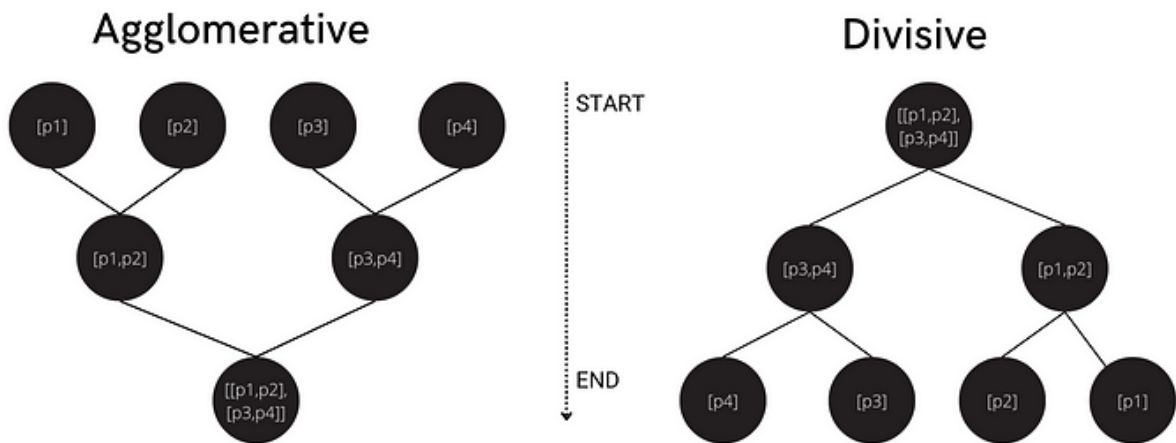


Quick Tip: If the elbow is vague, try the silhouette score to assess cluster separation.

Introduction to Hierarchical Clustering

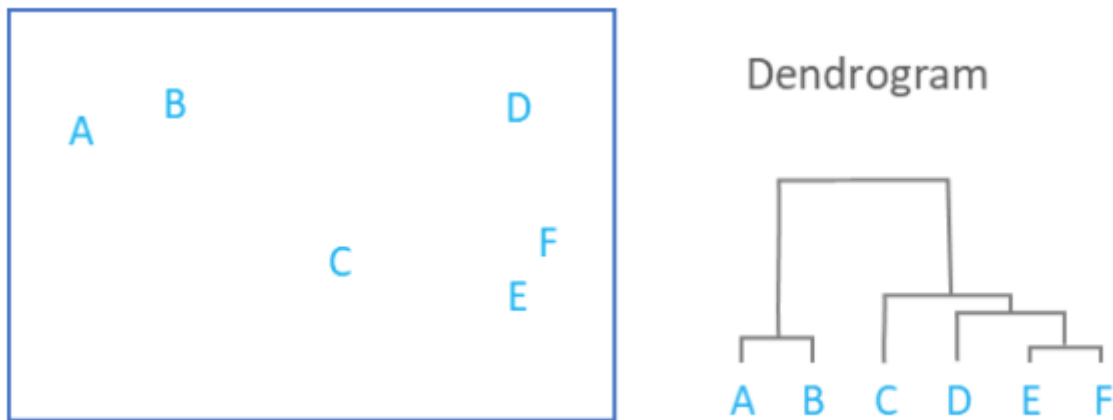
Hierarchical clustering creates a tree of clusters, like grouping friends by how close they are. It doesn't need **k** upfront and comes in two types:

- **Agglomerative:** Starts with each point as a cluster, merging the closest pairs step-by-step.
- **Divisive:** Begins with one cluster containing all points, splitting recursively.

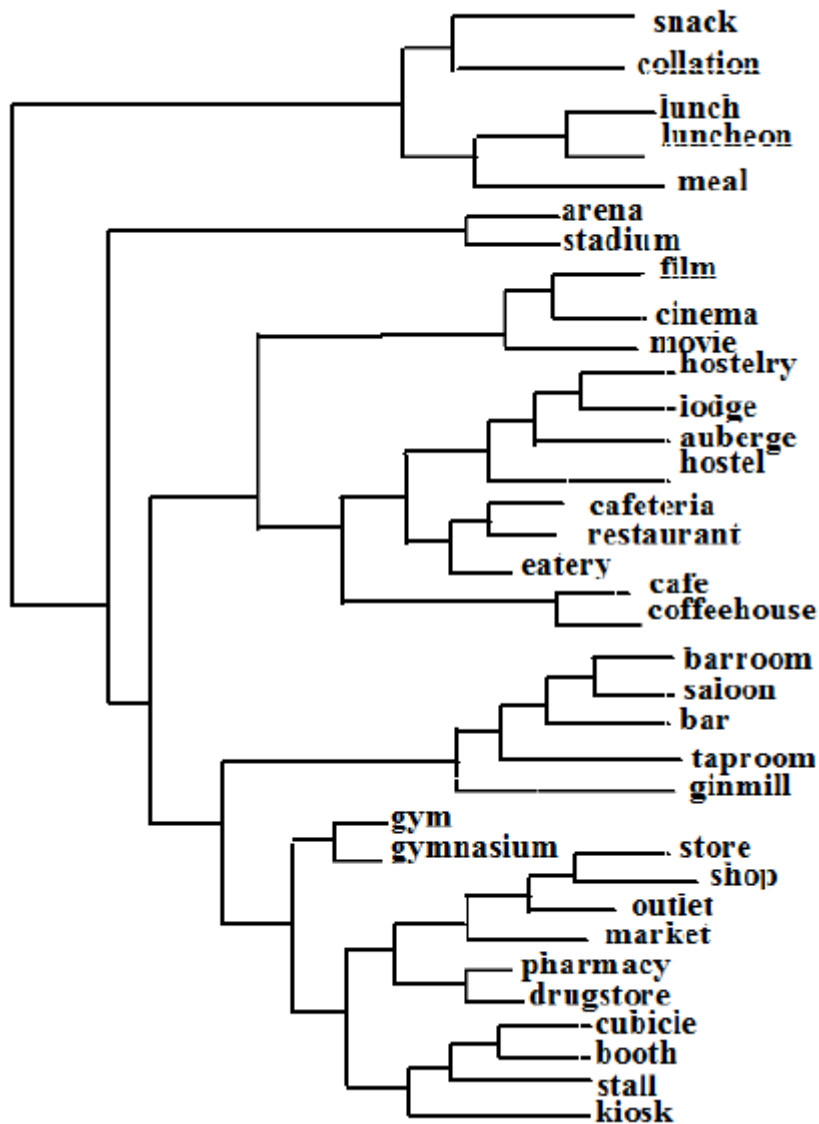


Understanding Dendrograms

A **dendrogram** shows the merging process, with branch heights reflecting cluster distances based on the chosen linkage. Cutting the dendrogram at different heights yields different numbers of clusters:



- **How to Read It:** Higher cuts give fewer clusters; lower cuts give more.
- **Why It's Useful:** Visualizes nested relationships and helps choose cluster granularity.



The distance between points is often **Euclidean**:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Types of Hierarchical Clustering

In hierarchical clustering, the **linkage criterion** determines how distances between clusters are measured, shaping how the algorithm groups data. The method you choose affects the resulting cluster shapes, sensitivity to outliers, and computational efficiency.

Single Linkage measures the distance between the closest points in two clusters:

$$d(C_a, C_b) = \min_{x \in C_a, y \in C_b} \|x - y\|$$

This method is useful for identifying elongated or chain-like structures in data. However, it is highly sensitive to noise and can lead to “chaining,” where distant points are linked through intermediate steps.

Complete Linkage looks at the distance between the farthest points in two clusters:

$$d(C_a, C_b) = \max_{x \in C_a, y \in C_b} \|x - y\|$$

It tends to form compact, well-separated clusters, making it a good choice when tight grouping is desired. The downside is its sensitivity to outliers, which can distort maximum distances.

Average Linkage calculates the average distance between all point pairs across two clusters:

$$d(C_a, C_b) = \frac{1}{|C_a||C_b|} \sum_{x \in C_a} \sum_{y \in C_b} \|x - y\|$$

This approach strikes a balance between single and complete linkage, offering more stable results. However, it can become computationally intensive, especially with large datasets.

Ward's Linkage merges clusters by minimizing the increase in within-cluster variance:

$$d(C_a, C_b) = \sqrt{\frac{|C_a||C_b|}{|C_a| + |C_b|}} \|\mu_a - \mu_b\|^2$$

It is particularly effective when clusters are spherical and similar in size. This method often produces balanced, compact clusters but may not perform well if the data contains uneven group sizes or shapes.

Try It: If you're clustering cities based on geographic location, which method would work best—single linkage, which connects the nearest neighbors, or complete linkage, which favors compact grouping? Reflect on your reasoning before continuing.

Comparing K-Means and Hierarchical Clustering

Both **K-Means** and **Hierarchical Clustering** are foundational unsupervised learning techniques used to group data points based on similarity. While they serve the same goal—clustering—they differ significantly in approach, scalability, and use cases.

Aspect	K-Means Clustering	Hierarchical Clustering
How It Works	Partitions the dataset into a predefined number k of clusters using centroid-based iteration	Constructs a hierarchy (dendrogram) by recursively merging or splitting clusters
Advantages	- Efficient on large datasets - Performs well with compact, spherical clusters	- No need to predefine k - Reveals multi-level structure - Supports various linkage methods
Limitations	- Requires specifying k in advance - Sensitive to outliers and initialization	- Computationally expensive for large datasets - Can be sensitive to noise

Aspect	K-Means Clustering	Hierarchical Clustering
Best Suited For	Customer segmentation, image compression, anomaly detection in high-volume data	Gene expression analysis, hierarchical document clustering, social network analysis

When to Choose Which?

Selecting between **K-Means** and **Hierarchical Clustering** depends on the dataset size, the desired output, and the nature of relationships within the data.

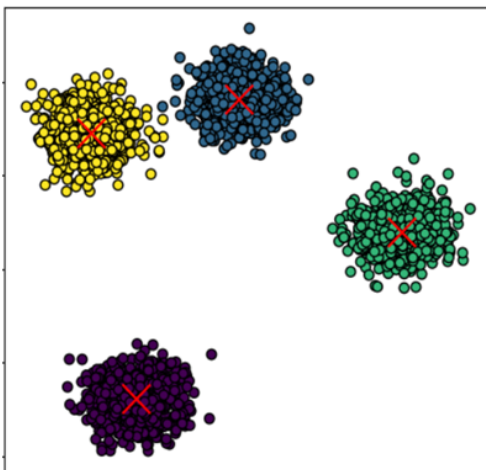
K-Means Clustering: Ideal when working with **large datasets** where speed and scalability are priorities. It performs well when the expected clusters are **roughly spherical and equally sized**.

Use Case: Grouping customers by purchasing behavior for targeted marketing campaigns.

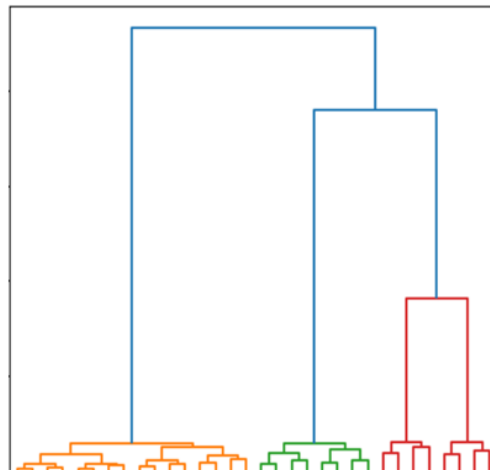
Hierarchical Clustering: Preferred when the focus is on **understanding nested or hierarchical relationships** between data points. It does not require specifying the number of clusters upfront and supports different linkage strategies (e.g., **Ward's method** for minimizing variance within clusters).

Use Case: Clustering biological species based on genetic similarity or organizing documents by topic hierarchy.

K-Means Clustering



Hierarchical Clustering



Fun Activity: Imagine clustering your playlist. Would k-means group songs by genre, or would hierarchical clustering with single linkage show how subgenres connect? Write down your choice!

Conclusion

You've unlocked the power of clustering! You can **apply k-means** to group data efficiently, using the elbow method and WCSS to pick **k**. You've mastered **hierarchical clustering**, exploring linkage types like single, complete, average, and Ward's, and interpreting dendrograms to reveal data

relationships. With code, math, and a comparison of both methods, you're equipped to choose the right tool for any clustering task. Keep experimenting to uncover the stories your data holds!